Develop a "Library Management System" to demonstrate Object-Oriented Programming concepts.
Part A: Classes and Objects
Create a class named Book containing:

* Book ID
* Book Name
* Author
* Price

Steps:
Start the program.
Create the Book class with book ID, name, author, and price.
Use a default constructor and parameterized constructor to initialize book details.
Use private variables with getters and setters to achieve encapsulation.
Create Person, Student, and Faculty classes to demonstrate inheritance.
Create the Area class and overload the area() method for square, rectangle, and circle.
Create Vehicle, Car, and Bike classes to demonstrate method overriding.
Create the abstract Shape class and implement it using Circle and Rectangle.
Create the Printable interface and implement it using the Report class.
Create objects in the main() method.
Call the required methods and display the results.
Stop the program.

Algorithm:
Step 1: Start.
Step 2: Create Book objects using default and parameterized constructors.
Step 3: Set and get book details using setter and getter methods.
Step 4: Create Student and Faculty objects using inheritance from Person.
Step 5: Create an Area object and call overloaded area() methods.
Step 6: Create Car and Bike objects using Vehicle reference and demonstrate method overriding.
Step 7: Create Circle and Rectangle objects using the abstract Shape class and call draw().
Step 8: Create a Report object and call the print() method from the Printable interface.
Step 9: Display all results.
Step 10: Stop.

UML Diagram:
```
+----------------+
        |     Book       |
        +----------------+
        | bookId         |
        | bookName       |
```

```
| author         |
| price          |
+----------------+
| constructors   |
| getters/setters|
| displayBook()  |
+----------------+


+----------------+
|    Person      |
+----------------+
       △
 +-----+-----+
 |           |
+-----------+ +-----------+
|  Student  | |  Faculty  |
+-----------+ +-----------+


+----------------+
|     Area       |
+----------------+
| area()         |
+----------------+


+----------------+
|    Vehicle     |
+----------------+
       △
 +-----+-----+
 |           |
+------+    +------+
| Car  |    | Bike |
+------+    +------+


+----------------+
|     Shape      |
|   <<abstract>> |
+----------------+
       △
 +-----+-----+
```

```
      |           |
+---------+  +-----------+
| Circle  |  | Rectangle |
+---------+  +-----------+
```

```
<<interface>>
+-------------+
|  Printable  |
+-------------+
      △
      |
+-------------+
|   Report    |
+-------------+
```

Code:

```
class Book {
   private int bookId;
   private String bookName;
   private String author;
   private double price;

   // Default Constructor
   Book() {
      bookId = 0;
      bookName = "Unknown";
      author = "Unknown";
      price = 0;
   }

   // Parameterized Constructor
   Book(int bookId, String bookName, String author, double price) {
      this.bookId = bookId;
      this.bookName = bookName;
      this.author = author;
      this.price = price;
   }

   // Getters
   public int getBookId() {
      return bookId;
   }
```

```java
    public String getBookName() {
        return bookName;
    }

    public String getAuthor() {
        return author;
    }

    public double getPrice() {
        return price;
    }

    // Setters
    public void setBookId(int bookId) {
        this.bookId = bookId;
    }

    public void setBookName(String bookName) {
        this.bookName = bookName;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void displayBook() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

// Inheritance
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
```

```
        this.age = age;
    }
}

class Student extends Person {
    int rollNo;

    Student(String name, int age, int rollNo) {
        super(name, age);
        this.rollNo = rollNo;
    }

    void display() {
        System.out.println("Student Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Roll No : " + rollNo);
    }
}

class Faculty extends Person {
    String subject;

    Faculty(String name, int age, String subject) {
        super(name, age);
        this.subject = subject;
    }

    void display() {
        System.out.println("Faculty Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Subject : " + subject);
    }
}

// Polymorphism - Method Overloading
class Area {

    void area(int side) {
        System.out.println("Area of Square = " + (side * side));
    }

    void area(int length, int breadth) {
        System.out.println("Area of Rectangle = " + (length * breadth));
    }
```

```java
    void area(double radius) {
        System.out.println("Area of Circle = " + (3.14 * radius * radius));
    }
}

// Polymorphism - Method Overriding
class Vehicle {
    void display() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    void display() {
        System.out.println("This is a Car");
    }
}

class Bike extends Vehicle {
    void display() {
        System.out.println("This is a Bike");
    }
}

// Abstraction
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

// Interface
interface Printable {
```

```java
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing Report");
    }
}

public class LibraryManagementSystem {

    public static void main(String[] args) {

        System.out.println("===== PART A & B =====");

        Book b1 = new Book();
        Book b2 = new Book(101, "Java Programming", "James Gosling", 550);

        b1.setBookId(100);
        b1.setBookName("Python");
        b1.setAuthor("Guido");
        b1.setPrice(450);

        b1.displayBook();

        System.out.println();

        b2.displayBook();

        System.out.println("\n===== PART C =====");

        Student s = new Student("Manasa", 19, 25);
        s.display();

        System.out.println();

        Faculty f = new Faculty("Ramesh", 40, "Java");
        f.display();

        System.out.println("\n===== PART D =====");

        Area obj = new Area();
        obj.area(5);
        obj.area(10, 5);
```

```
        obj.area(7.0);

        Vehicle v;

        v = new Car();
        v.display();

        v = new Bike();
        v.display();

        System.out.println("\n===== PART E =====");

        Shape c = new Circle();
        c.draw();

        Shape r = new Rectangle();
        r.draw();

        Report rep = new Report();
        rep.print();
    }
}
```

Output: